



Создай нейросетевую модель мифологической техносферы по поиску

Да — для **нейросетевой модели мифологической техносферы по поиску** лучше всего подходит гибридный граф знаний мифологического мира + нейронный энкодер запросов + графовый ретривер + ранжирование путей ответа. Такой дизайн согласуется с современными KG-RAG и graph-retrieval подходами, где поиск строится не по плоскому тексту, а по сущностям, связям и релевантным подграфам. ^{[1] [2] [3]}

Архитектура

Базовая схема может быть такой: пользовательский запрос кодируется текстовым энкодером; затем из графа мифологической техносферы извлекаются ближайшие сущности, связанные реликты, фракции, регионы и силы; после этого графовая нейросеть или path-ranker оценивает релевантность путей и возвращает ответ вместе с объяснимой структурой связей. ^{[1] [2] [4]} Такой подход особенно хорош для вашего мира, потому что он умеет искать не только по словам, но и по **архетипическим отношениям** вроде “реликт принадлежит архетипу”, “фракция действует в регионе”, “сила разрушает слой памяти”, “божество рождается из поля”. ^{[3] [5]}

Концепт данных

Минимальный граф мифологической техносферы может содержать такие типы узлов:

- Relic — реликты, например Дискета Памяти или Киноплётка Судьбы.
- Archetype — Память, Взгляд, Длительность, Порог, Судьба.
- Region — Архивные Пустоши, Долина Экранов, Река Перемотки.
- Faction — Архивариусы, Проекторы, Восстановители, Ткачи Смысла, Оракулы Сети.
- Force — Забвение, Шум, Фрагментация, Профанация.
- Entity верхнего уровня — Ноосферный Ум или Божество Сети.

Связи между ними можно задать как HAS_ARCHETYPE, LOCATED_IN, OPPOSES, RESTORES, RESONATES_WITH, TRANSFORMS_INTO, PREDICTS, THREATENS. ^{[2] [4]}

Python-скелет

Ниже — компактная, но уже рабочая **архитектурная заготовка** такой модели поиска. Это не обученная production-система, а концепт нейросетевого движка для вашего мира.

```
from dataclasses import dataclass
from typing import List, Dict, Tuple
import numpy as np
import networkx as nx
```

```

@dataclass
class Node:
    id: str
    kind: str
    text: str

@dataclass
class Edge:
    src: str
    rel: str
    dst: str
    weight: float = 1.0

class MythicTechnosphereSearch:
    def __init__(self):
        self.graph = nx.MultiDiGraph()
        self.embeddings: Dict[str, np.ndarray] = {}

    def add_node(self, node: Node):
        self.graph.add_node(node.id, kind=node.kind, text=node.text)

    def add_edge(self, edge: Edge):
        self.graph.add_edge(edge.src, edge.dst, rel=edge.rel, weight=edge.weight)

    def encode_text(self, text: str) -> np.ndarray:
        rng = np.random.default_rng(abs(hash(text)) % (2**32))
        vec = rng.normal(size=128)
        return vec / (np.linalg.norm(vec) + 1e-9)

    def build_embeddings(self):
        for node_id, attrs in self.graph.nodes(data=True):
            self.embeddings[node_id] = self.encode_text(
                f"{attrs['kind']} :: {attrs['text']}"
            )

    def retrieve_nodes(self, query: str, top_k: int = 5) -> List[Tuple[str, float]]:
        q = self.encode_text(query)
        scores = []
        for node_id, vec in self.embeddings.items():
            score = float(np.dot(q, vec))
            scores.append((node_id, score))
        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]

    def expand_subgraph(self, seeds: List[str], hops: int = 2) -> nx.MultiDiGraph:
        visited = set(seeds)
        frontier = set(seeds)

        for _ in range(hops):
            new_frontier = set()
            for node in frontier:
                neighbors = list(self.graph.successors(node)) + list(self.graph.predecessors(node))
                for neighbor in neighbors:
                    if neighbor not in visited:
                        new_frontier.add(neighbor)
            frontier = new_frontier

        return self.graph.subgraph(new_frontier)

```

```

        for nbr in neighbors:
            if nbr not in visited:
                visited.add(nbr)
                new_frontier.add(nbr)
        frontier = new_frontier

    return self.graph.subgraph(visited).copy()

def rank_paths(self, query: str, subgraph: nx.MultiDiGraph) -> List[Dict]:
    q = self.encode_text(query)
    ranked = []

    for u, v, data in subgraph.edges(data=True):
        text_u = self.graph.nodes[u]["text"]
        text_v = self.graph.nodes[v]["text"]
        rel = data["rel"]

        path_text = f"{text_u} -> {rel} -> {text_v}"
        p = self.encode_text(path_text)
        score = float(np.dot(q, p)) * data.get("weight", 1.0)

        ranked.append({
            "source": u,
            "relation": rel,
            "target": v,
            "score": score,
            "explanation": path_text
        })

    ranked.sort(key=lambda x: x["score"], reverse=True)
    return ranked[:10]

def search(self, query: str) -> Dict:
    top_nodes = self.retrieve_nodes(query)
    seed_ids = [node_id for node_id, _ in top_nodes]
    subgraph = self.expand_subgraph(seed_ids, hops=2)
    top_paths = self.rank_paths(query, subgraph)

    return {
        "query": query,
        "seed_nodes": top_nodes,
        "top_paths": top_paths
    }

def build_demo_world() -> MythicTechnosphereSearch:
    model = MythicTechnosphereSearch()

    nodes = [
        Node("relic_diskette", "Relic", "Дискета Памяти хранит архив утраченных посл
        Node("relic_film", "Relic", "Киноплёнка Судьбы переплетает повторение, утрат
        Node("region_archive", "Region", "Архивные Пустоши усиливают древнюю память
        Node("region_screens", "Region", "Долина Экранов концентрирует коллективный
        Node("faction_archivists", "Faction", "Архивариусы сохраняют реликты и восста
        Node("force_oblivion", "Force", "Забвение разрушает память, преемственность
        Node("deity_noos", "Entity", "Планетарный Ум рождается из согласованной памя

```

```

    Node("arch_memory", "Archetype", "Архетип Памяти собирает следы прошлого в у
]

edges = [
    Edge("relic_diskette", "HAS_ARCHETYPE", "arch_memory", 1.2),
    Edge("relic_diskette", "LOCATED_IN", "region_archive", 1.0),
    Edge("relic_film", "LOCATED_IN", "region_screens", 0.9),
    Edge("faction_archivists", "RESTORES", "relic_diskette", 1.1),
    Edge("force_oblivion", "THREATENS", "region_archive", 1.3),
    Edge("force_oblivion", "THREATENS", "arch_memory", 1.4),
    Edge("arch_memory", "TRANSFORMS_INTO", "deity_noos", 1.0),
    Edge("faction_archivists", "OPPOSES", "force_oblivion", 1.2),
    Edge("region_archive", "RESONATES_WITH", "arch_memory", 1.1),
]

for n in nodes:
    model.add_node(n)
for e in edges:
    model.add_edge(e)

model.build_embeddings()
return model

if __name__ == "__main__":
    engine = build_demo_world()
    result = engine.search("Как в мифологической техносфере победить забвение и усили
print(result)

```

Как сделать её нейросетевой по-настоящему

Чтобы это было не просто символическое хэш-эмбединг-ядро, а реальная нейросетевая система, замените `encode_text()` на `sentence-transformer` или `domain-adapted encoder`, а поверх графа используйте GNN для `message passing` и обучения на парах “запрос -> релевантные узлы/пути”. Современные работы по `graph retrieval` и `KG-RAG` как раз показывают, что граф помогает лучше отбирать сущности, отношения и пути рассуждения для ответа. ^{[2] [4] [3]}

Практический pipeline

Рабочий pipeline может быть таким:

1. Собрать мифологический `knowledge graph` мира. ^[5]
2. Закодировать тексты узлов и связей в эмбединги. ^[1]
3. На запросе искать `seed`-узлы по векторной близости. ^[1]
4. Расширять подграф на 1–3 hops и ранжировать пути. ^[4]
5. Передавать `top paths` в генератор ответа или `rule-based oracle`. ^{[2] [3]}

Для вашего мира

Для вашей задачи это особенно удобно, потому что поиск может отвечать не только на прямые запросы вроде “где находится Киноплёнка Судьбы”, но и на смысловые, например: “какие силы мешают рождению Планетарного Ума” или “какая фракция лучше всего восстанавливает память в Архивных Пустошах”. Такие вопросы естественно выражаются через графовые связи и path-ranking, а не через обычный keyword search.^{[1] [2]}

Могу следующим сообщением сразу дать **версию 2.0 в полноценном Python-коде**: с PyTorch, PyG/NetworkX, обучаемым ранжированием и демонстрацией поиска по вашему мифологическому миру.^{[6] [2]}

✱✱

1. <https://jeremilevesque.com/reviews/retrieval-augmented-generation-with-knowledge-graphs/>
2. <https://openreview.net/forum?id=ZikTuGY28C>
3. <https://neurips.cc/virtual/2024/poster/95524>
4. <https://www.nature.com/articles/s41598-025-21222-z>
5. <https://arxiv.org/abs/2510.20345>
6. <https://www.sciencedirect.com/science/article/abs/pii/S0950705124014448>
7. <https://www.nature.com/articles/s41598-025-30012-6>
8. https://www.youtube.com/watch?v=N2Ou_90UNbk
9. <https://openreview.net/forum?id=49flu0yDJ4>
10. <https://dl.acm.org/doi/10.1145/3777378>